

Software Requirements Specification (SRS) – Sprint 1

Recommender System Project

1. Architecture Overview

The system follows a modular architecture designed for scalability and maintainability.

Components:

- **Data Layer**
 - Input: products_clean.parquet, user_behavior.parquet
 - Stores structured product data and user interaction logs
- **ETL Layer**
 - Loads processed data
 - Generates interaction scores based on user behavior
 - Validates schema and ensures data consistency
- **Processing Layer**
 - Builds user-item interaction matrix
 - Handles data transformation for model consumption
- **Model Layer**
 - Popularity-Based Recommender
 - Collaborative Filtering (User-Based & Item-Based)
 - Matrix Factorization (ALS)
- **Evaluation Layer**
 - Computes metrics such as Precision@K, Recall@K, MAP@K, RMSE
- **Presentation Layer**
 - Maps product IDs to product names
 - Displays final recommendations

2. Data Governance & Privacy Standards

Data Handling:

- Only anonymized user IDs are used (no personally identifiable information)
- Data is stored in structured .parquet format for efficiency and integrity

Privacy Principles:

- No storage of sensitive personal data (e.g., name, email, location)
- User interactions are used solely for recommendation purposes

- Data access is restricted to authorized processes only

Compliance Considerations:

- Follows basic principles aligned with GDPR-like practices:
 - Data minimization
 - Purpose limitation
 - Secure storage

3. Security Controls

Data Security:

- Read-only access to input datasets
- Validation checks to prevent malformed data ingestion

Application Security:

- Input validation for all user and product IDs
- Protection against invalid queries or missing data

System-Level Controls:

- Local execution environment with restricted file access
- No external API exposure in current sprint

4. MLOps Strategy Outline

Model Lifecycle:

- Data -> Training -> Evaluation -> Deployment (manual pipeline for Sprint 1)

Versioning:

- Models are versioned implicitly through code structure
- Future scope includes:
 - Model version tracking
 - Experiment logging

Monitoring (Planned):

- Track model performance metrics over time
- Detect degradation in recommendation quality

Automation (Future Scope):

- Integration with tools like Airflow/Prefect for pipeline automation
- Scheduled retraining based on new data

5. Deployment Constraints

Current Constraints:

- System runs in a local environment (no cloud deployment yet)
- No real-time recommendation API
- Batch processing only

Performance Limitations:

- Designed for moderate dataset size
- Memory-dependent due to in-memory matrix operations

Scalability Constraints:

- Not optimized for distributed systems
- No horizontal scaling in current implementation

Future Improvements:

- Deploy via FastAPI for real-time recommendations
- Use distributed frameworks (Spark) for large-scale data
- Containerization using Docker

6. Summary

This system implements a foundational recommender architecture using structured behavioral data. It supports multiple recommendation strategies and provides a base for future enhancements in scalability, automation, and deployment.
